Understanding Software Engineering through Team Project

Managing Complexity with Rules and Tools in a Team as a Professional Problem Solver

ASE 285

ASE @ Northern Kentucky University

Meet Tim

Tim is a sophomore at NKU ASE program, and he needs to attends ASE 285, and he was told that in this course, "team project" is an important part of the course, and it mimics real-world software engineering process.

> **Tim**
>
> "Hello, I'm Tim; I'm excited to start the team project, but at the same time, I'm not sure what exactly is the team project is about; what's the goal and what should I do."

Tim is in the state of "Unknown unknowns", in other words, he doesn't know what he doensn't know; so, he's afraid of what is coming to us, but at the same time he's excited to challenge the unknowns.

Chapter 1: Understanding Software Engineering

Tim remembers the meaning of software engineering and software engineers that he read from the GitHub ASE repository and discussed in class.

- Software engineering is about managing complexity.
- Software engineers are problem solvers who manage complexity.
- To manage complexity and solve problems, software engineers use rules and tools.

Tim scratches his head. These concepts seems too abstract.

> **Tim**
>
> “Wait… I thought software engineering was just about writing code to make apps and games. I can already make a simple calculator app in Python!”

🐍 **Python**

```python
# Tim's calculator - "This works perfectly!"
def calc():
    a = input("First number: ")
    b = input("Second number: ")
    op = input("Operation (+,-,*,/): ")
    if op == "+": print(float(a) + float(b))
    elif op == "-": print(float(a) - float(b))
    # ... and so on
```

Tim's Innocent Belief

Maybe this is a common idea shared by most amateur programmers.

Tim

"If it runs, it's good software engineering. Right?"

He keeps asking questions.

Tim

"So why all this talk about 'managing complexity'? Isn't coding just... coding?

And 'problem solving'? That sounds like solving math puzzles, not programming.

Plus, I have VSCode and Node.js. Why do I need 'rules'? My code runs fine without any rules!"

Julie Helps Tim

Then, Julie, an ASE senior, passes by. Seeing Tim's confused face, she stops to help.

Tim

"I don't understand this definition of software engineering as 'managing complexity.' I made a calculator for homework using Python in VSCode - it was pretty easy!"

Julie

"Oh, nice! How many lines of code were in your calculator?"

Tim

"About 100 lines."

The Scale of Real Projects

Julie

"Perfect! Now imagine this..." **pulls out her laptop** "Here's my recent team project for one of the ASE course - a food delivery app. Guess how many lines?"

Tim

"Maybe... 1,000?"

Julie

laughs "Try 20,000 lines. And that's just the backend! In the real world, even a simple project can have hundreds of thousands of lines of code. A modern OS has about 20–30 million."

Tim

"WHAT?!"

Julie

“And it’s not just about size. Let me show you what happened when we DIDN’T manage complexity…”

Chaos When There are No Rules

We start simple, but we add complexity!

```
# Week 1: Tim's style - "It works!"
user1_order = {"food": "pizza", "price": 10}
user2_order = {"food": "burger", "price": 8}
# ... copy-paste for 100 users
```

```
# Week 4: Chaos begins
user1_order = {"food": "pizza", "price": 10, "address": "123
Main"}
user2_order = {"food": "burger", "cost": 8}  # Oops! 'cost'
not 'price'
# ... now need to update 100 places
```

```
# Week 8: Complete disaster
# Different team members used different formats:
orders = [
    {"item": "pizza", "price": 10, "user_id": 1},
    {"food_item": "burger", "amount": 8, "customer": 2},
    {"meal": "pasta", "total": 12.5, "userId": "three"}  #
string ID?!
]
```

Tim

"Oh... that's messy."

Team Collaboration Chaos

Julie

"Exactly! Now imagine 5 people working on this. Sarah adds delivery tracking, Mike adds payment processing, Lisa adds restaurant management… all without rules."

Tim

"It would be chaos!"

Julie

"That's where 'managing complexity' comes in. Watch this:"

With Software Engineering Principles and Design

🐍 Python

```
# With Software Engineering principles:

from dataclasses import dataclass
from typing import List, Optional
from datetime import datetime

@dataclass
class Order:
    """Single source of truth for order structure"""
    order_id: int
    user_id: int
    items: List['OrderItem']
    total_price: float
    status: str
    created_at: datetime
    delivery_address: Optional[str] = None

class OrderService:
    """Centralized order management - everyone uses this"""
    def create_order(self, user_id: int, items: List[dict]) -
> Order:
        # Validation, business logic, consistency guaranteed
        pass
```

Following the Rules

Tim

"Oh! It's way better. So everyone has to follow the same structure, I mean, follow the rules?"

Julie

"Exactly! And it gets more complex. What about:"

- "Payment failures?"
- "Restaurant is closed?"
- "User wants a refund?"
- "Database crashes?"
- "10,000 users ordering at once?"

Tim

"I... never thought about all that."

The Real Challenge

Julie

"Your 100-line calculator handles maybe 5 requirements. Our app handles 50+ requirements. Without rules and tools, it's impossible to manage."

Tim

"So that's why we need software engineering! It's not about making code work once, it's about making it work always, for everyone, even when things go wrong!"

Julie

"Now you're getting it! And don't forget - everything keeps changing; clients want new features, team members come and go, libraries get updated, security threats emerge… the list never ends."

Tim's Realization

Tim

"Wow, it's like juggling while riding a bicycle!"

Julie

"Perfect analogy! And software engineering gives you the skills to keep all those balls in the air."

Tim

"Now I think I understand why software engineers are problem solvers who manage complexity."

Julie

"Exactly! Software engineers must choose the right rules and tools for each problem; there's no one-size-fits-all solution. Also, they should follow the common rules for all projects."

The Journey Ahead

Tim

"I really want to learn more about software engineering!"

Julie

"In ASE courses, you'll learn these rules and tools through real projects. Trust me, when your team project grows from 100 to 10,000 lines, you'll be grateful for every principle you learned. The pain of not knowing them... **shudders** ...I've been there."

Tim

"Thanks, Julie! I'm actually excited about the challenges, and I am also happy to understand the importance of the project now!"

Chapter 2:
The Team Project Begins

Form teams of 3-4 people (5 only in a truly special case), and choose a team lead.

The goal of a team project is **Practice of Problem Solving** in a **Team**.

- **The Problem:** “Make a to-do list web application”
- **Required Tools:** Node.js (JavaScript), MongoDB/Mongoose, Git/GitHub, VSCode, Markdown
- **Process:** The Agile process

Team Formation

After some awkward mingling, Tim finds himself at a table with:

Ken

"Hey! I did a chat app project last semester. Learned a lot from our mistakes… mostly the hard way."

Jane

"I'm Jane! I love front-end design but I've never worked in a team project before."

Tim

"I'm Tim. Just learned that software engineering is about managing complexity. Ready to experience it firsthand, I guess!"

Selecting a Team Leader

Jane

"What is the team leader's role?"

Ken

"It's mainly managing projects. The team leader should manage schedule, report weekly progress, manage tests, manage documents, and evaluate team members."

Tim

"I'm not familiar with managing schedules and weekly progress reports."

Ken

"You'll learn about them as soon as we start the agile process. Anyway, we should select a team leader."

Team Leader Selection Rule

1. When students attend other ASE courses, they are not allowed to be in the same team in the courses unless they have a good reason to be in the same team in these classes.
2. When a student already did the role of a project leader, it is highly recommended that the student does not lead the team unless the team have a good reason; the team is lead by a student who has not lead a team before.
3. Read related document, or ask the professor, to understand the requirements and expectations.

Jane

"Ken, as you already finished a team project before, how about leading the team this time?"

Ken

pauses "Well... last time I was just a team member. But sure, I can try! I learned what NOT to do, at least."

Tim

"I'm okay with that. We will support you!"

Jane

"Great! So what's our first step, leader?"

Ken

pulls out notebook "Right! First, we need to understand what we're building. A to-do list app."

Doing Project in the age of AI

Tim

"Well, a to-do list app sounds simple to me. I'm thinking we finish in 1 week, then polish for the rest of the semester. Easy A!"

Jane

"I don't know if it's OK or not, but I think we can finish the whole project without coding or anything with the help of AI."

Ken

"I understand what you are saying. But, if we do that, I'm sure we are wasting our time and money, and even worse, future opportunities."

Tim

"I don't understand. We're doing what's required - building a to-do app. We should get full points according to the grading rubric."

Ken

"You're right about the points. But think about it - if you can only do what AI can easily do, why would companies hire you when they can use AI for almost free?"

Jane

"Oh… that's a scary thought."

Ken

“Exactly. In my last project, we also thought about using AI for everything. But you know what? The team member who actually did that learned nothing. When they went to interviews, they couldn’t explain their own code!”

Jane

“Software engineers are smart, so they can see instantly who did the real project, or nothing.”

Ken

“Yes. In doing project, you will also see who does use AI without understanding anything. Nobody wants to work with software engineer who cannot solve any problems without AI.”

Jane

"I agree. We must use AI only for helping us, not to replace us when it comes to solving problems."

Tim

"So you're saying we need to **develop skills that AI can't replace**?"

Ken

"Right! AI is a tool, not a replacement for understanding or programming. We should use it to learn faster and to be a better problem solver, not to avoid learning or solving problems."

Jane

"Like using a calculator in math class - you still need to understand the concepts."

Ken

“Perfect analogy! So, let’s use AI as our assistant, not our ghost writer. Also, when we use AI, we write down the usage of AI under what circumstances. Deal?”

Team

“Deal!”

AI Usage Guideline

The Skills that AI Can't Replace:

1. Coming up with a solution to a given problem
2. Communication Skills
3. Plan ahead and follow the plan
4. Understanding stakeholder needs and translating them to technical requirements
5. Making ethical decisions and considering social impact
6. Team conflict resolution and emotional intelligence
7. Evaluating trade-offs between different solutions
8. Taking responsibility and accountability for decisions

The Skills that We Can Do Better with AI

1. Code review - AI can spot patterns we might miss
2. Discussing error messages to fix bugs
3. Learning new technologies faster through interactive Q&A
4. Exploring alternative implementations
5. Understanding best practices and design patterns
6. Refactoring code for better quality
7. Making Prototype and MVP (Minimum Viable Product)

The Skills that We Can Delegate to AI

1. Making tests *(but we should understand and verify them!)*, including the creation of mock data for testing
2. Making documents *(but we need to review and customize)*, including the generation of API documentation from code
3. Writing boilerplate code
4. Initial code scaffolding and project setup
5. Syntax checking and formatting
6. Anywork that can be automized

Important Note

Even for “delegatable” tasks, we should:

- Understand what AI generated
- Verify the quality and correctness
- Customize for our specific needs
- Learn from the AI’s approach
- Make a record of our progress and learning

The Portfolio - The Goal of Project

Ken

“The main goal of each project is to build a strong portfolio that demonstrates the problems we’ve solved as problem solvers.”

Tim

“What is a portfolio?”

Ken

“You know what a resume is, right? It’s like a one-page summary of who you are - education, skills, experience. But it just lists things.”

Tim

“Yeah, like ‘Proficient in Python, Java, React’…”

Ken

“Exactly! But here’s the thing - everyone writes that. How do employers know if you’re really good at problem solving?”

Jane

“That’s where the portfolio comes in?”

Ken

“Bingo! A portfolio SHOWS what you can do, not just tells. Look at this example:”

Resume vs. Portfolio

📄 RESUME says:

- "Experienced in full-stack development"
- "Team player with leadership skills"
- "Problem-solving abilities"

💼 PORTFOLIO shows:

- "Here's the e-commerce site I built: [https://nkuase.github.com/myportfolio] → It solved inventory tracking issues for 50+ small businesses"
- "I led a 4-person team to build a study app
  → See our Git history, my code reviews, and sprint planning docs"
- "We faced a 10-second load time problem
  → I optimized it to 0.5 seconds using caching and lazy loading
  → Here's my technical blog post explaining the solution

Tim

“Oh! So resume is the advertisement, portfolio is the proof!”

Ken

“Perfect analogy! And for software engineers, a good portfolio includes:”

1. **GitHub repositories** - Clean, documented code with meaningful commit messages
2. **Live projects** - Deployed apps people can actually use
3. **Technical write-ups** - Blog posts explaining your problem-solving journey
4. **Contribution history** - Shows consistent work and collaboration skills

Jane

"So our to-do app could be part of our portfolio?"

Ken

"Yes, it SHOULD be! But only if we make it portfolio-worthy. That means:"

- **Real problems solved** (not just basic Create-Read-Update-Delete operations)
- **Unique features** that show innovation or creative thinking
- **Clean, professional code** that others can understand
- **Comprehensive documentation** including README, API docs, and user guides
- **Solid test coverage** showing we care about quality
- **A compelling story** about challenges we overcame

Tim

"I see. In short, we should prove that we're effective problem solvers through this project."

Jane

"Now I see why we shouldn't just let AI do everything. An AI-generated project wouldn't have OUR problem-solving story!"

Ken

"Exactly! Employers want to see how YOU think, how YOU solve problems, how YOU work in a team. That's what gets you hired."

Tim

"This is eye-opening. If I ever become a team leader and need to hire someone, I'd never choose people without problem-solving skills. A project portfolio would be the perfect screening tool!"

Ken

"You're thinking like a future tech lead already! Remember: anyone can learn syntax and frameworks from tutorials. But learning to identify problems, design solutions, and work with others? That's what separates engineers from coders."

Jane

"So this ASE course isn't just about building apps..."

Ken

“It’s about building professional problem-solvers. The apps are just the evidence.”

GitHub, GitHub.io and LinkedIn

Use these three platforms to build and share your portfolio:

1. **GitHub** - Showcase your projects with code, tests, docs, and insights about the problem and your solution.
2. **GitHub.io** - A free personal webpage to organize and present your projects professionally.
3. **LinkedIn** - The main platform recruiters use; create a profile and link your GitHub and GitHub.io pages.

Even Further - Published Applications

These are part of good portfolio:

1. Published apps

- **iOS** applications published on Apple Store
- **Android** applications published on Google Store
- **PC** applications published on Microsoft Store

2. Published web application services

- **Vercel**: Great for Next.js, React apps
- **Netlify**: Perfect for static sites
- **Heroku**: Traditional web apps

Most of the services provide a free plan

- Both Vercel & Netflify provide a free plan.
- Heroku discontinued the free plan, but there are alternatives such as Render.

Features and Requirements

Ken

"Now, we need to understand WHY we want to build this application."

Tim

"Hmm, we already know **what** to do - making a to-do app. I can find a ton of implementations in all programming languages. Can't I just copy any of them? I think it's a good learning experience."

Ken

"Technologically yes, but without the **why**, we may end up with a portfolio that nobody finds interesting."

Jane

“I don’t get it. A to-do app is a to-do app, right?”

Ken

“Let me show you something…” **pulls out his phone**

“I have 5 to-do apps installed. Guess how many I actually use?”

Team

“All of them?”

Ken

“Zero. They all failed to solve MY actual problem.”

The Problem Discovery

Ken

"Let's do an experiment. What problems do YOU face with managing tasks?"

Tim

"Well... I always forget homework deadlines because they're scattered across different course websites."

Jane

"I make to-do lists but never look at them again. Out of sight, out of mind!"

Ken

"For me, I can't prioritize. Everything seems urgent, so I get paralyzed."

Ken

“See? Three people, three different problems. Most to-do apps just let you add and check off tasks. They don’t solve OUR problems.”

From Generic to Specific

Ken

"Let's analyze existing apps and their target users:"

```
Generic To-Do App:
- Add task ✓
- Mark complete ✓
- Delete task ✓
- Maybe categories ✓
Target: Everyone (= No one specifically)

Todoist:
- Natural language input ("Tomorrow at 2pm")
- Project hierarchies
- Team collaboration
- Karma points for motivation
Target: Productivity enthusiasts & teams

Microsoft To-Do:
- Integrates with Outlook
- "My Day" planning
- Shared grocery lists
Target: Microsoft ecosystem users & families
```

Tim

"Oh! Each successful app solves specific problems for specific people!"

Finding Our Niche

Jane

"So who are WE building for?"

Ken

"Great question! Let's brainstorm. Who do we understand best?"

Tim

"Well... ourselves? College students?"

Ken

"Perfect starting point! What's unique about college student task management?"

College Student Task Problems

The team starts listing:

```
1. Multiple course websites (Canvas, Blackboard, professor's sites)
2. Different deadline formats (11:59 PM vs midnight vs in-class)
3. Group project coordination chaos
4. Balancing study/work/social/health
5. Procrastination and all-nighters
6. Forgetting which assignments are worth more points
7. "Did I submit that or just dream I did?"
8. Syllabus changes mid-semester
9. Study group scheduling nightmares
```

Jane

"Wow, we have real problems that generic apps don't address!"

The Requirements Born from Pain

Ken

"Now, let's turn these problems into features. But remember - we can't solve everything in one semester."

Tim

"How do we choose?"

Ken

"By talking to potential users and finding the MOST painful problem."

Jane

"Let's survey our classmates!"

Back to Reality - Classroom Constraints

Tim

"This survey idea sounds great, but... we only have one week for planning and Professor Cho recommended we need to start making a prototype now."

Jane

"Plus, everyone's busy with the homework assignments."

Ken

"You're right. Welcome to the difference between ideal software engineering and classroom reality!"

Ken

"However, in real companies, they usually spend months on user research. They have budgets for surveys, focus groups, user interviews..."

Tim

"But we're just students with a semester project."

Ken

"Exactly! So we adapt. Here's what we CAN do in our constraints:"

Jane

"So what do we do?"

Ken

"We brainstorm! It's not perfect, but it's WAY better than building blindly."

Tim

“How do we brainstorm effectively?”

Quick Requirements Discovery Session

> **Ken**
>
> “Simple process - 30 minutes max, but we can have another session of brainstorm if necessary. The key is to start somewhere and iterate, not to get stuck in endless planning.”

```
1. Personal Pain Points (10 min)
   - Each member lists 5 problems they ACTUALLY face.
   - Be specific and honest
   - No judging yet

2. Validation Check (10 min)
   - "Do others have this problem too?"
   - Quick poll in team Discord/Slack
   - Ask roommates, friends nearby
   - Even 3-4 confirmations = validated

3. Pick ONE Core Problem (10 min)
   - Which can we solve in 10 weeks?
   - Which would WE actually use?
   - Which shows technical skills?
```

The Results are In

[After brainstorming]

Ken

"The results are in. The top 3 pain points:"

```
1. Assignment Deadline Anxiety (18/20 students)
   - "I wake up panicked thinking I missed something"
   - "I check Canvas 10 times a day"
   - "I missed a quiz worth 20% because it was hidden in announcements"

2. Priority Confusion (15/20 students)
   - "All my professors think their class is the only one"
   - "I spent 10 hours on a 1% assignment, missed a 15% quiz"
   - "I can't tell what's actually important"

3. Submission Paranoia (12/20 students)
   - "Did I actually submit or just upload?"
   - "Which version did I submit?"
   - "Was it on time?"
```

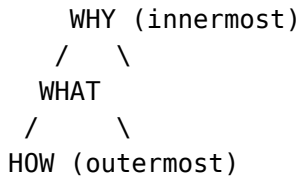
Tim

"So our to-do app should solve THESE specific problems!"

The Golden Circle of Software Engineering

> **Ken**
>
> “Before we dive into features, let me share something important I learned from my internship.”

Draws three circles on whiteboard



> **Ken**
>
> “This is the Golden Circle. Most teams start from the outside - HOW to build. But successful software starts from WHY.”

Tim

"I don't get it. Isn't it obvious? We build a to-do app because… it's the assignment?"

Ken

"That's exactly the trap! Watch this:"

```
❌ Starting with HOW (Outside-in):
"Let's use React and Node.js"
"We'll have user authentication"
"We'll use PostgreSQL for the database"
→ Result: Generic app #1,000,000 (nobody uses this app)

✅ Starting with WHY (Inside-out):
"Students are missing deadlines and it's affecting their grades".
"Why? They're overwhelmed managing multiple platforms"
"What would help? Unified deadline visibility"
"How? Canvas integration, smart notifications..."
→ Result: A solution people actually need
```

> **Ken**
>
> “Don’t forget that software engineers are problem solvers, not programmers or coders.”

The AI Reality Check

Jane

"But can't we just ask ChatGPT to design our app?"

Ken

"Great question! Let me show you something..." **Opens ChatGPT**

Ken's prompt: "Build a to-do app for college students"

ChatGPT: "Here's a full-stack to-do app with user authentication, task categories, due dates..."

Jane

"That looks complete!"

Ken

"Now watch this:"

Ken's prompt: "Why do college students at NKU struggle with assignments despite having access to Canvas, planners, and existing to-do apps?"

ChatGPT: *Generic response about time management and organization*

Ken

"See? AI can tell us HOW to build and WHAT features to include. But it can't tell us WHY our specific users are struggling. That requires human understanding."

The Three Questions

Ken

"As software engineers, we must answer three questions IN ORDER:"

```
1. WHY (Only humans can answer):
   - Why is this problem worth solving?
   - Why do current solutions fail (or not satisfactory)?
   - Why would someone use this software (even when they should pay for it)?

2. WHAT (Humans decide, AI can suggest):
   - What is the core problem? (not symptoms)
   - What is the minimum solution (MVP or Minimum Viable Product)?
   - What does successful result look like?

3. HOW (AI shines here):
   - How to implement the features?
   - How to build the system?
   - How to write efficient code?
```

Tim

"So AI is like a master carpenter who can build anything, but WE need to know what's worth building?"

Ken

"Perfect analogy!"

Practical Example

Jane

“Let’s try this with our project!”

Team Exercise:

```
1. WHY (Our human insight):
"NKU students are failing classes not because they're lazy,
but because Canvas buries important deadlines in different
sections. We know this because WE experience it daily."

2. WHAT (Our decision, with AI help):
Human: "The core problem is deadline visibility"
AI suggested: Calendar, notifications, integrations
Human decided: "Focus on deadline extraction first"

3. HOW (AI assists):
"ChatGPT, how do we integrate with Canvas API?"
"GitHub Copilot, implement a deadline parser"
"Claude, read my notification algorithm and give me feedback"
```

Tim

“I see! We’re not replaced by AI, we’re directors using AI as our crew!”

The Engineering Mindset

Ken

"This is why software engineering isn't just coding or programming. Look at the difference:"

```
CODER's (or PROGRAMMER's) Approach:
- Opens editor
- Starts coding
- Asks AI for implementation
- Builds features
- Hopes someone uses it

ENGINEER's Approach:
- Understands the problem (WHY)
- Validates with users (WHY matters)
- Defines core solution (WHAT)
- Then leverages AI for building (HOW)
- Measures if it solved the problem
```

Jane

"No wonder so many apps fail. They skip the WHY!"

Tim

"And that's probably why employers value problem-solving over coding skills..."

Ken

“Exactly! Anyone (or any AI) can code. But understanding human problems and translating them into solutions? That’s irreplaceable.”

The Takeaway

Prof. Cho

“Excellent discussion! Remember:”

“AI can write code better than most humans. But it can’t sit with a stressed student at 2 AM to understand why they’re crying over missed deadlines. It can’t feel the frustration of juggling 5 different course websites. That empathy and understanding - that’s your superpower as software engineers.”

Team

Silent moment of realization.

Prof. Cho

“So yes, use AI for the HOW. Let it help with the WHAT. But the WHY? That’s your job. That’s what makes you real software engineers and problem solvers, not typists.”

Tim

"I think I understand the meaning of software engineering as 'managing complexity' and software engineers as 'problem solvers' better now. The complexity isn't just about code size - it's also about understanding human problems!"

Jane

"And I know how to be a problem solver who can use AI as a powerful tool for coding, not as a magic box that does my homework. We direct it, not the other way around."

Ken

"Welcome to real software engineering, team! Let's solve problems together, and it should be fun … and of course, rewarding in the end."

Chapter 3:
Projects in Practice

From **Unknown Unknowns** to **Known Knowns** through **Known Unknowns**.

- Iterative process with two sprints
- Building prototypes and MVPs
- Understanding roles and responsibilities
- Following professional team rules

The Unknown Territory

The team sits in their first planning meeting, notebooks open, coffee in hand.

Ken

"Alright team, we have our goal: build a to-do app that solves deadline anxiety for college students. But..."

Tim

"But what?"

Ken

"We're in what's called 'unknown unknowns' territory."

Jane

"Unknown... unknowns? That sounds like a riddle!"

The Four Stages of Knowledge

Ken

"It's actually a framework for understanding what we know. Let me draw this out:"

```
1. Unknown Unknowns (Most Dangerous)
   → "We don't even know what we don't know"
   → Example: We think building is easy, unaware of 50+ problems ahead

2. Known Unknowns (Progress!)
   → "We know what we need to learn"
   → Example: We realize we need to learn Canvas API integration

3. Known Knowns (Expertise)
   → "We know it and can do it"
   → Example: We can implement authentication confidently

4. Unknown Knowns (Intuition)
   → "We know it but don't realize we know it"
   → Example: Good coding practices we do automatically
```

Tim

"So right now, we're at stage 1? We don't even know what problems we'll face?"

Ken

"Exactly! And that's terrifying because we can't plan for problems we don't know exist."

Jane

"How do we escape this trap?"

Ken

"By moving as quickly as possible from unknown unknowns to known unknowns. Once we know what we don't know, we can make a real plan."

The Two-Sprint Strategy

Ken

"That's why we use an iterative process with two sprints:"

```
Sprint 1: Discovery & MVP (from Week 4 to the first midterm)
Goal: Discover the main unknown unknowns
Deliverable: Minimum Viable Product (MVP)
Success metric: "Now we know what we're dealing with"

Sprint 2: Refinement & Completion (after the first midterm to Week 16)
Goal: Build the full solution based on what we learned
Deliverable: Final product with polish
Success metric: "It works well and we're proud to show it"
```

Tim

"So Sprint 1 isn't about building the perfect app?"

Ken

"Nope! Sprint 1 is about learning. We build the minimum possible thing that works, and through that process, we discover all the problems."

Prototypes vs MVPs

Jane

"Wait, I keep hearing two terms: prototype and MVP. Are they the same?"

Ken

"Great question! They're actually very different:"

Understanding Prototypes

```
Prototype:
- Purpose: Answer "Can we even do this?"
- Quality: Rough, hacky, throwaway code
- Testing: None (it's temporary)
- Documentation: Minimal or none
- Design: Whatever works fastest
- Timeline: Days to one week
- Example: "Let me spend 2 days seeing if Canvas API works"
```

> **Ken**
>
> “Prototypes are for when you’ve never done something before. You’re just checking feasibility.”

Understanding MVPs

```
MVP (Minimum Viable Product):
- Purpose: "This actually works and users can try it"
- Quality: Production-ready code
- Testing: Full test coverage
- Documentation: Complete
- Design: Proper architecture
- Timeline: Weeks
- Example: "A working to-do app with core features"
```

> **Ken**
>
> "The MVP is what we deliver at the end of Sprint 1. It's real software that works, just with minimal features."

Tim

“So if I understand correctly: prototype is ‘Can we do it?’ and MVP is ‘Here’s the simplest version that works’?”

Ken

“Perfect! And here’s the key: we only build prototypes when we’re truly uncertain. If we’ve built something similar before, we skip straight to MVP.”

Jane

“For our project, do we need prototypes?”

Ken

“Let’s discuss. Have any of us built a web app with Node.js and MongoDB before?”

Team

Silence, then head shakes

Ken

“Then yes, we might need a quick prototype for the basic tech stack. But since the professor gave us example code, we can probably start with MVP directly.”

Why Aim for High Quality?

Tim

"I have a question. Why do we need 'high quality' if we're just students? Can't we just make it work and move on?"

Ken

"That's the amateur mindset talking. Let me tell you what I learned from my last project..."

The Last Project Story

Ken

“Last semester, my team thought exactly like you. ‘Let’s just make it work, we’ll fix it later.‘ So we wrote code fast, no tests, no documentation, messy structure.“

Jane

“What happened?”

Ken

“Week 8: We wanted to add a new feature. It took 3 days because we had to understand our own messy code first.”

Ken

“Week 10: Bug appeared. Spent 2 days finding it because we had no tests.”

Ken

"Week 12: Team member left. New person joined. Took them a week to understand the code because we had no documentation."

Ken

"By the end, we spent more time fighting our own code than building features. If we had started with quality, we would have finished faster AND had a better product."

Tim

"I see... quality isn't just about looking professional. It's about making our own lives easier!"

Ken

"Exactly! As professional software engineers, we aim for:"

- Software that is easy to **find and fix bugs**
- Software that is easy to **add features**

Focus on Real Problems

Ken

"And here's the beautiful part: when your architecture is clean, you can focus all your energy on solving real problems instead of fighting technical debt."

Jane

"What do you mean by 'real problems'?"

Ken

"Think about it. If your code is a mess, where does your time go?"

Tim

"Understanding the code... fixing bugs... figuring out where to add new features..."

Ken

"Exactly! All your time goes to managing the mess. But with clean code:"

- "You spend 5 minutes understanding the code, not 5 hours"
- "You spend time making the app better for users"
- "You spend time solving the actual deadline anxiety problem"

Roles and Responsibilities

The team decides to document their roles clearly.

Jane

"Ken, you mentioned roles before. Can we clarify who does what?"

Ken

"Great idea! Let me explain how this mirrors real software engineering teams..."

Professor as Manager

Ken

"Professor Cho is playing the role of a Manager. In real companies, managers:"

- Manage people and politics
- Report to higher management
- Evaluate team leads and members
- Support teams in problem-solving
- Make career decisions (promotions, raises, hiring)

Tim

"So Professor Cho evaluates us like a real manager would?"

Ken

"Exactly. The difference is that Professor Cho can't fire us!" **laughs** "But the evaluation process is similar to real-world performance reviews."

Team Leaders as Tech Leads

Ken

"I'm the team leader, which means I'm playing the Tech Lead role. Tech leads:"

- Manage team complexity and organize work
- Report progress to managers
- Set goals and schedules
- Make sure everyone is moving forward
- Manage artifacts (code, tests, documents)
- Preside over weekly meetings
- Lead presentations

Tim

"That's a lot of responsibility!"

Ken

"It is! But it's great practice for real jobs. Tech leads are the bridge between management and engineers."

Team Members as Senior Engineers

Ken

"You two, Tim and Jane, are team members. In our project, you're playing senior-level software engineers. Senior engineers:"

- Tackle complexity through design and implementation
- Design architecture and data models
- Build modules and interfaces
- Write code and tests
- Raise issues early
- Mentor others (in real teams)

Jane

“Wait, why are we ‘senior’ engineers? We’re students!”

Ken

“Because in the age of AI, there are practically no junior-level software engineering jobs anymore.”

Tim

“What do you mean?”

Ken

“Junior engineers used to do basic tasks like writing simple functions, copying code patterns, or doing repetitive work. Now AI can do all that.”

Ken

“So when companies hire today, they expect everyone to be problem solvers from day one. That’s what ‘senior-level’ means - you can solve problems independently.”

Professional Team Rules

Ken

"Before we start working, we need to establish some professional ground rules. These are the same rules real software engineering teams follow."

Jane

"Like what?"

Ken

"Four critical rules: No surprises, Show no emotions, Be professional, and Understand others first."

Rule 1: No Surprises

Ken

"In professional teams, unexpected problems break trust more than difficult problems."

Tim

"I don't understand. Isn't a problem just a problem?"

Ken

"Let me give you an example. Which is worse:"

"Scenario A: On Monday, you tell me you're stuck on authentication and need help."

"Scenario B: On Friday, when the feature is due, you suddenly reveal you never started authentication."

Tim

“Oh… Scenario B is way worse! Because now there’s no time to fix it!”

Ken

“Exactly! And worse than that - I lose trust in you. I can’t plan properly if you don’t communicate.”

Jane

“So ‘no surprises’ means…”

Ken

“It means:”

- Communicate early when issues appear
- Be transparent about progress and blockers
- Set expectations clearly
- Never let anyone be blindsided

Rule 2: Show No Emotions

Tim

“Wait, we’re not allowed to have emotions? That seems harsh!”

Ken

“No, no - you can have emotions! The rule is about not letting emotions control your communication.”

Jane

“Can you give an example?”

Ken

"Sure. Let's say you're frustrated because the code isn't working:"

```
❌ Emotional Response:
"This stupid MongoDB keeps breaking! I hate this project!
Nothing works! I give up!"

✅ Professional Response:
"I'm encountering connection errors with MongoDB.
I've tried X and Y. Could someone review my connection code?"
```

Ken

"See the difference? The second approach:"

- Focuses on facts, not feelings
- Provides specific information
- Asks for help constructively
- Maintains team morale

Tim

“So it’s not about hiding emotions, it’s about staying professional even when stressed?”

Ken

“Exactly! Professionalism means staying calm under pressure. It builds trust and helps the team solve problems faster.”

Rule 3: Be a Professional

Jane

"What does 'be a professional' mean exactly? Isn't that just... everything?"

Ken

"It's about actions and results, not titles. Being professional means:"

- Deliver on promises, on time
- Respect others with fairness and courtesy
- Take responsibility for mistakes
- Strive for quality in every task

Tim

"So if I promise to finish a feature by Tuesday, I should finish it by Tuesday?"

Ken

"Yes! Or if you realize you won't make it, you communicate that EARLY. Remember Rule 1: No surprises."

Jane

"And if we make a mistake?"

Ken

"Own it. Say 'I made a mistake in the data model' instead of blaming tools, circumstances, or others. Then fix it and learn."

Rule 4: Understand Others Before Being Understood

Tim

“This one sounds… philosophical?”

Ken

“It’s actually the most practical rule! Let me show you why.”

Ken

“Imagine you and I have a disagreement about how to structure the database:”

```
❌ Bad Approach:
Tim: "We should use separate collections!"
Ken: "No, we need embedded documents!"
Tim: "But separate collections are better!"
Ken: "Embedded documents are the right way!"
→ Result: Argument, no progress, hurt feelings

✅ Good Approach:
Tim: "I'm thinking separate collections. What's your concern?"
Ken: "I'm worried about query performance. What problem
      are you trying to solve?"
Tim: "I want easier updates. Could we test both approaches?"
→ Result: Understanding, collaboration, better solution
```

Ken

"When you understand others first:"

- You learn the real problem
- You build trust faster
- You find better solutions
- You avoid pointless conflicts

Jane

"So it's about listening deeply before trying to convince others?"

Ken

"Exactly! Stephen Covey said it best: 'Seek first to understand, then to be understood.'"

Putting It All Together

Tim

"Okay, so we have:"

- Two sprints to move from unknown unknowns to known knowns
- Build MVP in Sprint 1, polish in Sprint 2
- Aim for high quality to make our lives easier
- Follow professional rules

Jane

"And we understand our roles as tech lead and senior engineers."

Ken

"Perfect! Now let's talk about the specific process we'll follow..."

The Weekly Rhythm

Ken

"Every week follows this pattern:"

```
Monday (First Meeting):
- I present our progress to Professor Cho
- Update everyone on team status
- Discuss any blockers

Tuesday-Sunday:
- Everyone works on their tasks
- Update progress pages on Canvas
- Communicate issues immediately
- Help each other when stuck
```

Jane

"What are these 'progress pages'?"

Ken

"They're Canvas pages where we track our work. Think of them as our public project diary:"

- What did you accomplish this week?
- What are you working on now?
- What blockers are you facing?
- What help do you need?

Tim

"Why do we need this? Can't we just use GitHub?"

Ken

"GitHub shows code changes. Progress pages show the full story: problems faced, decisions made, learning achieved."

Three Celebration Moments

Ken

"Throughout our project, we'll have three major milestones:"

```
1. Project Kickoff (Week 4)
   - Present our plan
   - Show our goals and timeline
   - Get feedback before starting

2. MVP Launch (after the first midterm)
   - Demo our working MVP
   - Discuss what we learned
   - Present Sprint 2 plan

3. Final Presentation (Week 16)
   - Show completed project
   - Celebrate achievements
   - Reflect on journey
```

Jane

"These sound like checkpoints to make sure we're on track!"

Ken

"Exactly! And they force us to have working software at each point, not just 'almost done' excuses."

Tim's Realization

Tim

"I just realized something. This whole structure - the sprints, the roles, the rules - it's all about managing complexity, isn't it?"

Ken

"Yes! Everything we've discussed is a tool for managing complexity:"

- Two sprints manage complexity of unknowns
- Clear roles manage complexity of teamwork
- Professional rules manage complexity of communication
- Quality code manages complexity of implementation

Jane

"And all these 'rules and tools' help us be better problem solvers!"

Ready to Begin

Ken

"Alright team, I think we're ready. We understand:"

- Where we are (unknown unknowns)
- Where we're going (known knowns)
- How we'll get there (two sprints)
- How we'll work together (professional rules)

Team

"Let's do this!"

Ken

"But first… we need to set up our development environment and understand the tools. That's our homework for this week!"

Tim

"Wait, we haven't even installed anything yet?"

Ken

laughs "Welcome to software engineering - you spend half your time setting up before you can actually build anything!"

Chapter 4:
Two Projects Structure

Understanding the complete project roadmap:

- Team project: Building TodoApp together
- Individual project: Creating TagGPT alone
- Weekly presentations and progress tracking
- Evaluation and portfolio building

Two Projects Ahead

The team gathers around Ken's laptop as he opens the course syllabus.

Ken

"Okay team, here's something important: we're not just doing one project this semester - we're doing two."

Tim

"Two projects?! I thought we're just building the to-do app?"

Ken

"That's the team project. But there's also an individual project later in the semester."

Jane

"What's the individual project about?"

The Two Projects

Ken

"Let me break it down:"

```
Team Project: TodoApp
- Work together as a team of 3-4 people
- Solve the 'Missing Deadline' problem
- Build a web application
- Tools: Node.js, MongoDB, Git, VSCode
- Timeline: Weeks 1-16 (whole semester)

Individual Project: TagGPT
- Work independently
- Use AI tools effectively
- Build something to showcase your skills
- Timeline: Starts in 2nd iteration
```

Tim

"Wait, so we'll be doing both at the same time in the second half?"

Ken

"Yes, but the team project takes priority in Sprint 1. Individual project becomes more active in Sprint 2."

Why Two Projects?

Jane

"Why do we need both? Isn't one project enough?"

Ken

"Great question. They teach different skills:"

Team Project: Collaboration Skills

```
Team Project teaches:
- Working with others (communication)
- Managing shared codebase (Git conflicts!)
- Dividing work effectively
- Coordinating schedules
- Dealing with different coding styles
- Team leadership (for team leader)
- Professional collaboration skills

Real-world parallel:
→ 90% of software engineering is team work
```

Individual Project: Personal Excellence

```
Individual Project teaches:
- Taking full ownership
- Making all decisions yourself
- Showing your unique skills
- Working with AI tools independently
- Building complete features alone
- Creating your personal portfolio piece

Real-world parallel:
→ Your portfolio proves YOUR capabilities
```

Ken

"Think of it this way: Team project proves you can work with others. Individual project proves you can work alone. Companies need both."

Tim

"Makes sense! So what happens each week?"

The Weekly Presentation Rhythm

Ken

“Every week follows this pattern. Let me walk you through it…”

Monday: First Class Meeting

```
What happens:
1. All team leaders present their progress
2. Professor Cho asks questions and gives feedback
3. Teams discuss blockers and get guidance
4. Leaders take notes for their team

Team Leader's role:
- Show progress from Canvas pages
- Demo any working features
- Explain what's next
- Raise any issues

Duration: short (it depends on the situation)
```

Jane

"So you have to present every single week?"

Ken

"Yes. It keeps everyone accountable and helps catch problems early."

Tim

"What if someone on the team made no progress?"

Ken

"Here's an important rule: I don't call people out publicly in presentations. I just report the overall team status."

Jane

"Why not? Wouldn't that push them to work harder?"

Understanding Leadership Boundaries

Ken

"Because there's a difference between technical leadership and people management. Let me explain:"

```
Tech Lead (My role):
- Manage technical work and progress
- Help solve technical problems
- Coordinate task distribution
- Report objective status

Manager (Professor Cho's role):
- Handle people issues
- Make hiring/firing decisions
- Evaluate personal performance
- Deal with conflicts or motivation problems
```

Ken

“If someone isn’t contributing, that’s a people problem. I report the facts to Professor Cho, and he handles it as a manager.”

Tim

“So you focus on ‘what needs to be done’ and Professor Cho focuses on ‘who’s doing it well’?”

Ken

“Exactly! This mirrors real companies. Tech leads shouldn’t try to do the manager’s job.”

Progress Pages: Your Public Diary

Jane

"You mentioned Canvas pages for tracking progress. How exactly does that work?"

Ken

"Every person gets their own progress page on Canvas. Think of it as your public project diary:"

Your Progress Page includes:

1. Current Sprint Goals
   - What you planned to achieve this sprint

2. Weekly Updates (update every week!)
   - What you accomplished this week
   - What you're working on now
   - Any blockers or challenges
   - Links to GitHub commits/PRs

3. Learning Log
   - New skills learned
   - Problems solved
   - AI usage documentation

4. Reflection
   - What went well
   - What could improve

Tim

"So it's like a blog about our project work?"

Ken

"Yes! And it serves multiple purposes:"

- Keeps the team informed
- Helps Professor Cho evaluate fairly
- Creates a record for your portfolio
- Shows your problem-solving journey

Jane

"And everyone can see everyone else's pages?"

Ken

"Yes. Transparency is key. If you're stuck, others can help. If you're making great progress, others can learn from you."

Three Celebration Moments

Ken

"Now, let me tell you about the best parts of the semester - our three celebration moments!"

Tim

"Celebration moments?"

Ken

"Yes! These are major milestones where we formally present our work and celebrate progress."

Celebration 1: Project Kickoff (Week 4)

```
What we present:
- Team introduction and roles
- Project vision and goals
- Feature list and requirements
- Sprint 1 plan with timeline
- Technology stack chosen
- How we'll use AI tools

What we submit:
- Detailed project plan (HW2)
- Individual goals on progress pages
- Team project page with links

Why it matters:
- Get feedback before building
- Ensure everyone's aligned
- Set clear expectations
```

Jane

“So this is like our project proposal?”

Ken

“Exactly! It’s where we say ‘Here’s what we’re going to build and here’s how we’ll do it.‘ Professor Cho and other teams can give us feedback.“

Tim

“What happens if Professor Cho thinks our plan is too ambitious?”

Ken

“That’s the point of the kickoff! Better to adjust the plan early than fail later. We might need to reduce features or change our approach.”

Celebration 2: MVP Launch (after the first midterm)

```
What we present:
- Live demo of working MVP
- What we learned in Sprint 1
- Challenges we overcame
- What changed from original plan
- Sprint 2 plan and goals

What we show:
- Working application (even if basic)
- GitHub repository with clean code
- Tests and documentation
- Progress page updates

Why it matters:
- Prove we can deliver working software
- Share lessons learned
- Get feedback for Sprint 2
```

Ken

“This is the most important milestone. Many students think Sprint 1 is just ‘practice’ and they’ll build everything in Sprint 2. That’s a huge mistake!”

Jane

“Why?”

Ken

“Because if you don’t have a working MVP by the first midterm, you’re basically saying ‘I’ve spent half the semester and have nothing to show.‘ No company would accept that.“

Tim

“So the MVP has to actually work, not just ‘almost work’?”

Ken

“Correct! It should be simple, but it should WORK. Users should be able to use it, even if it only has basic features.”

Celebration 3: Final Presentation (Week 16)

```
What we present:
- Complete project demo
- All features working
- Lessons learned (whole semester)
- What we're proud of
- What we'd do differently
- How this helps our portfolio

What we show:
- Polished application
- Comprehensive documentation
- Test coverage
- Deployment (if applicable)
- Portfolio-ready presentation

Why it matters:
- Showcase our final achievement
- Reflect on growth
- Celebrate success!
```

Jane

“This is like our graduation moment!”

Ken

“Exactly! And the best teams create presentations so good they can use them in job interviews.”

Tim

“Really? The final presentation becomes interview material?”

Ken

“Absolutely! Imagine telling an interviewer: ‘Here’s a project where I led a team to build a solution for 1,000+ students. Let me show you the demo and explain the technical decisions we made.‘“

Tools for Success

> **Ken**
>
> "Let's talk about the tools we'll use to track all this work:"

```
GitHub:
- Store all code and project files
- Track changes with commits
- Manage tasks with Issues
- Review code with Pull Requests
- Show project history

Canvas Pages:
- Track weekly progress
- Document learning journey
- Show individual contributions
- Link to GitHub artifacts

GitHub.io (optional):
- Professional project website
- Portfolio showcase
- Public project information
```

Jane

"Why use both GitHub and Canvas pages? Seems redundant."

Ken

"Different audiences and purposes:"

- GitHub shows WHAT you built (code)
- Canvas shows HOW you built it (process)
- GitHub is for developers
- Canvas is for evaluation and reflection

Evaluation: How We're Graded

Tim

"Okay, important question: how exactly are we evaluated?"

Ken

"Everything is based on rubrics. Professor Cho doesn't grade based on feelings - he grades based on whether you delivered what you promised."

Understanding Rubrics

```
Rubrics check:
✓ Did you deliver features promised in your plan?
✓ Is your code well-designed and tested?
✓ Did you update progress pages weekly?
✓ Did you present your work effectively?
✓ Is your system flexible and maintainable?
✓ Did you use AI tools appropriately?
✓ Did you contribute fairly to the team?

Evaluation is based on:
- Your progress page (continuous updates)
- Your GitHub contributions (commits, PRs)
- Your presentations (kickoff, MVP, final)
- Peer evaluation (team members rate each other)
- Quality of deliverables (code, tests, docs)
```

Jane

"What does 'quality' mean exactly?"

Ken

"As professional software engineers, low-quality results are not acceptable. Quality means:"

- Code is clean and follows best practices
- Tests exist and pass
- Documentation is clear and complete
- Design is thoughtful, not random
- Features actually work reliably

Tim

"So even if we build all the features, if the code is messy, we lose points?"

Ken

"Yes! Because messy code shows you're not thinking like a professional engineer. Remember: we're building portfolio pieces, not homework assignments."

Individual Responsibility

Ken

"Here's something critical to understand: even though this is a team project, each person is individually responsible for their results."

Jane

"What do you mean?"

Ken

"Let me give you two scenarios:"

```
Scenario A: Low Progress, But Communication
Student: "I'm stuck on authentication. I've tried X and Y.
         Can someone pair with me tomorrow?"
Result: Team helps, problem gets solved, shows good process

Scenario B: Low Progress, No Communication
Student: *silent for two weeks*
Week 8: "Sorry, I couldn't finish my part"
Result: Damages team, loses trust, earns poor evaluation
```

Ken

"The difference? In Scenario A, the student is being professional. They're stuck but communicating. In Scenario B, they're breaking the 'No Surprises' rule."

Tim

“So lack of progress is okay if we communicate?”

Ken

“Not exactly. Lack of progress still impacts your grade. BUT communicating shows professional behavior and gives the team a chance to help.”

Ken

“The worst situation is: low progress + no communication. That’s a double failure - technical AND professional.”

Warning: Last-Minute Sprints

Jane

"What if someone does nothing for weeks but then works really hard at the end and delivers great results?"

Ken

"That will earn less even if the final result looks good."

Tim

"Why? If the work is good, isn't that what matters?"

Ken

"Because it shows you can't manage time and it breaks trust. Let me explain..."

Why Last-Minute Sprints Fail

Problems with last-minute work:

1. Trust Damage
   - Team couldn't plan around you
   - Others had to carry extra burden
   - Manager (Professor) couldn't track progress

2. Quality Issues
   - No time for proper review
   - No time for testing
   - No time for iteration
   - Rushed code is often buggy

3. Learning Loss
   - You missed the journey
   - You couldn't help teammates
   - You didn't practice professional habits

4. Professional Impact
   - In real jobs, this behavior leads to firing
   - Wasting time is the most dangerous habit

Ken

"Remember: treat the deadline as FINAL. Set your own deadline at least one week earlier. If you plan to finish Week 8, target Week 7."

Jane

"That makes sense. Build in buffer time for problems!"

Ken

"Exactly! Professional software engineers always have buffers. They know things will go wrong."

Changes Are Allowed!

Tim

"What if we realize our plan was wrong and we need to change our goals?"

Ken

"Great question! Changes are absolutely allowed and expected. But they must be communicated immediately."

When plans change:

✓ Update your Canvas progress page immediately
✓ Explain WHY the change was necessary
✓ Show WHAT you're changing to
✓ Update team project page
✓ I'll discuss it in weekly presentation

Example:
"Week 5 Update: Changed from Canvas API integration to manual deadline input. Reason: Canvas API requires admin approval (3-week wait). New approach: users manually enter deadlines. This is better for MVP scope anyway."

Ken

“The key is: no surprises! If you change something, tell everyone. Don’t wait until the presentation to reveal major changes.”

Individual Project Presentations

Jane

"You mentioned individual project presentations. How do those work?"

Ken

"For the individual project, each person may present their work in class, depending on the course. Some courses use a peer evaluation (reviewing three assigned peers' projects) instead of, or together with, a presentation. When there is a presentation, it's different from team presentations:"

Individual Presentation:
- You schedule your own time slot
- You present alone (not as a team)
- Focus on YOUR problem-solving story
- Happens in 2nd iteration
- Keep it short (the time depends on the situation)

What to show:
- The problem you solved
- Your technical approach
- Challenges you overcame
- How you used AI tools
- What you learned
- Demo of working features

Know Your Audience

Tim

"How technical should our presentations be?"

Ken

"Excellent question! The biggest mistake students make is explaining everything in technical jargon."

Jane

"But we're presenting to Professor Cho, who's technical!"

Ken

"Yes, but imagine you're also presenting to:"

- A CEO who wants to know business value
- A product manager who cares about users
- Your future employer reviewing your portfolio

```
Bad Presentation:
"I used React hooks with useEffect and useState
to implement a MongoDB aggregation pipeline with
Mongoose schemas..."

Good Presentation:
"Students were missing deadlines, so I built a
system that automatically highlights urgent tasks.
Here's how it works: *demo*. The technical
challenge was integrating with Canvas, which I
solved by..."
```

Ken

“Start with the problem and value. Then go technical. Not the reverse!”

Presentation Best Practices

Ken

"A few tips I learned from my last project:"

```
✓ Number your slides (always!)
✓ Stay natural even if something breaks
✓ Focus on what audience needs to hear
✓ Be concise - respect everyone's time
✓ Practice your demo beforehand
✓ Have a backup plan if demo fails
✓ Show enthusiasm for your work
✓ Be ready to answer questions
```

Tim

"What if the demo breaks during the presentation?"

Ken

"You stay calm and professional. Show no emotions! Explain what should happen, show screenshots or video if possible, and move on."

Portfolio Building

Jane

"We keep talking about portfolios. How do these projects become portfolio pieces?"

Ken

"By the end of the semester, you should have at least two portfolio-ready projects:"

- Your team project contribution
- Your individual project

What makes a project "portfolio-ready":

✓ Solves a real, specific problem
✓ Has clean, documented code on GitHub
✓ Includes tests and proper design
✓ Has a README explaining the project
✓ Shows your problem-solving process
✓ Demonstrates technical skills
✓ Has a demo you can show
✓ Proves you can finish projects

How to showcase:
→ GitHub repository (public)
→ GitHub.io page (professional presentation)
→ LinkedIn post (share your achievement)
→ Resume bullet point (with link)

Tim

"So we're not just doing projects for grades - we're building our future career materials!"

Ken

"Exactly! Every ASE course should add at least two projects to your portfolio. After 4-5 ASE courses, you'll have 10+ projects to show employers."

Jane

"That's actually exciting! Each project is an investment in our future."

Professor Cho's Recommendations

Professor Cho drops by to check on the team's planning session.

Prof. Cho

"I couldn't help but overhear your discussion. Let me add a few recommendations from my 20 years in the industry:"

SEFE: Start Early to Finish Early

Prof. Cho

"This is the first rule. If you want to finish strong, start immediately. Don't wait for perfect understanding or perfect tools."

Tim

"But what if we don't know how to start?"

Prof. Cho

"Start anyway! Build a prototype. Make mistakes. Learn. The biggest enemy of good work is waiting for certainty that never comes."

Control What You Can

Prof. Cho

"You can't control:"

- Whether Canvas API approves your request
- Whether your teammate gets sick
- Whether MongoDB has a bug

"But you CAN control:"

- Your goals and milestones
- Your communication
- Your learning and effort
- Your backup plans

Prof. Cho

“Focus energy on what you control. Accept what you can’t. This is wisdom from ancient Stoic philosophers, and it applies perfectly to software engineering!”

Celebrate Small Wins

Prof. Cho

"Don't wait until Week 16 to feel good about your work. Celebrate every small victory:"

- First successful database connection? Celebrate!
- First test passing? Celebrate!
- First feature working? Celebrate!
- First code review completed? Celebrate!

Jane

"But isn't that silly? They're just small things."

Prof. Cho

"Big achievements are just collections of small wins stacked together. If you only celebrate the end, you'll be miserable for 15 weeks!"

Learn from Everything

> **Prof. Cho**
>
> “Treat homework, exams, quizzes - everything - as opportunities to practice problem-solving:”

```
❌ Student mindset:
"Ugh, another homework. Let me just finish it and
forget about it."

✅ Engineer mindset:
"This homework is practice for my project. What can
I learn here that will help me later?"
```

> **Prof. Cho**
>
> “The students who see everything as practice become the best engineers. The students who just chase grades learn nothing.”

Growth Over Points

Prof. Cho

"Final advice: focus on developing skills, not just earning grades."

Tim

"But grades matter for GPA and graduation!"

Prof. Cho

"True. But let me ask you: would you rather have:"

"Student A: 4.0 GPA, but can't solve real problems"

"Student B: 3.5 GPA, but has 10 impressive projects and strong problem-solving skills"

Ken

"Student B would get hired first!"

Prof. Cho

“Exactly! Focus on becoming an excellent problem solver. The grades will follow naturally.”

My Door Is Always Open

Prof. Cho

"Remember: I'm here to support you. If you're stuck, confused, or overwhelmed - just ask for help!"

Jane

"When should we ask for help?"

Prof. Cho

"Ask when:"

- You've tried for 2 hours and made no progress
- You're not sure if your approach is right
- You need help understanding a concept
- You have a team conflict
- You're falling behind and don't know why

Prof. Cho

"But please don't wait until Week 15 to tell me you've been stuck since Week 3!"

The Journey Begins

The team looks at each other with newfound understanding.

Tim

"So we have:"

- Two projects to build
- Clear weekly rhythm
- Three celebration moments
- Professional evaluation system
- Portfolio-building opportunities

Jane

"And we know exactly what's expected of us!"

Ken

"Plus we have all the support we need. Let's make these projects something we're proud to show employers!"

Team

“Let’s do this!”

Prof. Cho

“I’m excited to see what you’ll build. Remember: this is not just about grades. This is about becoming professional software engineers who can solve real problems!”